

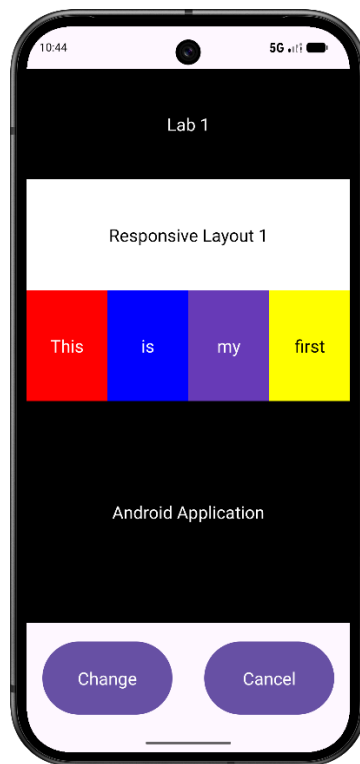
Practical 1: Creating Responsive Android User Interfaces

Objective

In this lab, you will practice building responsive user interfaces in Android Studio using two different layout techniques: `LinearLayout` with weights and `ConstraintLayout`. Finally, you will learn how to initialize version control and link your local Android project to a remote repository on GitHub or GitLab.

Part 1 – LinearLayout with Weights

The following is the target user interface that you must recreate in Android Studio.



Requirements

Create the interface using XML code.

You must:

1. Use a `LinearLayout` for all the layouts (parent and inner).
2. Use an appropriate `android:orientation`.
3. Use `layout_weight` to distribute available space.
4. Use `0dp` for the dimension corresponding to the weighted direction.
5. Use `match_parent` and `wrap_content` appropriately.
6. Include all required `TextViews`.

7. Include the two buttons:
 - Change
 - Cancel
8. Make the text appropriately positioned within each `TextView`.
9. Avoid using fixed pixel (px) dimensions.

Important

*Do NOT use the drag-and-drop feature of the Android Studio Design panel to create the UI.
You must:*

- *open the XML/code editor;*
- *add the required views yourself;*
- *write the XML attributes yourself;*
- *determine the appropriate weights and dimensions yourself.*

You are expected to understand why each XML attribute is being used.

Part 2 – ConstraintLayout with Weights

Create a second version of the same user interface (as a separate android app), but this time use `ConstraintLayout` with weights.

The appearance of the application should be as close as possible to the interface created in Part 1.

Requirements

Your `ConstraintLayout` version should:

1. Use `ConstraintLayout` as the root and inner layouts.
2. Add all `TextViews` and `Buttons` manually through XML.
3. Use constraints to position the views.
4. Use constraints relative to:
 - the parent;
 - other views where appropriate.
5. Avoid unnecessary fixed dimensions.
6. Use `wrap_content` and `match_parent` where appropriate.
7. Use appropriate margins where necessary.

Suggested approach

Do not try to reproduce the interface by calculating exact pixel positions.

The goal is to create a layout that can grow, shrink and rearrange itself when the available screen space changes, which is the central idea of responsive layouts covered in the lecture.

Converting Between Layout Types

For this practical, you are allowed to use the Android Studio Design panel when changing the parent layout.

For example, if your current layout is a `ConstraintLayout` and you want to convert it to a `LinearLayout`:

- a. Click the `Design` tab.
- b. In the `Component Tree`, right-click `main`.
- c. Select `Convert view...`
- d. Select `LinearLayout`.
- e. Modify the resulting XML as necessary.

You may use this feature only to change/convert the parent layout.

You must NOT:

- *drag `TextViews` onto the screen;*
- *drag `Buttons` onto the screen;*
- *use the `Design` panel to construct the UI;*
- *rely on the visual editor to automatically generate the constraints.*

The purpose of this practical is for you to practise writing and understanding Android XML layouts.

Part 3 – GitHub/GitLab Repository

Create a repository for your Android project using GitHub or GitLab.

You may use either:

- GitHub
- GitLab

Step 1 – Create the Repository

Create a new repository with the name: `[student ID]-ResponsiveLayout1`

Step 2 – Connect Android Studio to the Repository

Connect your Android Studio project to your GitHub/GitLab repository.

Your project should be under version control.

You should be able to see your Git repository and commit history from Android Studio.

Step 3 – Commit Your Work

Make meaningful commits as you develop the application.

For example:

- `First commit, initial Android project.`
- `Added all TextViews of the UI.`
- `Added the two buttons "Change" and "Cancel".`
- `Completed the layout.`

Avoid commit messages such as:

- `update`
- `stuff`
- `final final`

Step 4 – Push the Project

Push your project to GitHub/GitLab.

Your remote repository should contain the Android Studio project required to open and build the application.

Submission Requirements

Before finishing the lab, check that:

Part 1 – LinearLayout

- UI recreated using `LinearLayout`.
- `layout_weight` used appropriately.
- Appropriate `0dp` dimensions used with weights.
- `match_parent` / `wrap_content` used appropriately.
- No unnecessary hard-coded pixel dimensions.
- All `TextViews` are present.
- `Change` button is present.
- `Cancel` button is present.

Part 2 – ConstraintLayout

- Same UI recreated using `ConstraintLayout`.
- Views positioned using constraints.
- Constraints are relative to the parent and/or other views.
- No unnecessary fixed positioning.
- UI remains usable at different screen sizes.

Part 3 – GitHub/GitLab

- Repository created.
- Android Studio project connected to repository.
- Meaningful commits made.
- Project pushed to GitHub/GitLab.
- Repository contains the completed project.

Important Rules

1. No Drag-and-Drop UI Construction

Do not use the Design panel's drag-and-drop functionality to create your UI.
You are expected to write the XML yourself.

2. Design Panel Conversion Is Allowed

You may use:

Design → Component Tree → Right-click main → Convert view...

when changing the parent layout between layouts such as `ConstraintLayout` and `LinearLayout`.

3. Avoid Hard-Coded Sizes

Do not design the application for only one particular phone.

For example, avoid:

```
android:layout_width="350px"
```

Instead, use appropriate responsive approaches such as:

```
android:layout_width="match_parent"
```

or:

```
android:layout_width="wrap_content"
```

4. Test Your Application

Run your application and test it on an emulator/device.

If possible, test it using more than one screen size or orientation.

Ask yourself:

Does my UI still look reasonable when the available space changes?

That is the central goal of this practical.